APRIL-MAY 2026

Large Language Models & Generative AI

From Pytorch to Frameworks

Giulia Coucourde

University of Turin · giulia.coucourde@unito.it

Retrieval Augmented Generation

What is RAG?

Retrieving relevant data and generating accurate, context-aware responses to improve AI outputs.

R	Retrieve	Find useful information
A	Augment	Add it to the AI's knowledge
G	Generate	Create a better response

RAG

Without RAG: question → LLM → answer

The model answers from training data only. *Risk*: outdated, hallucinated, or wrong answers.

With RAG: question → retrieve relevant documents → LLM → answer

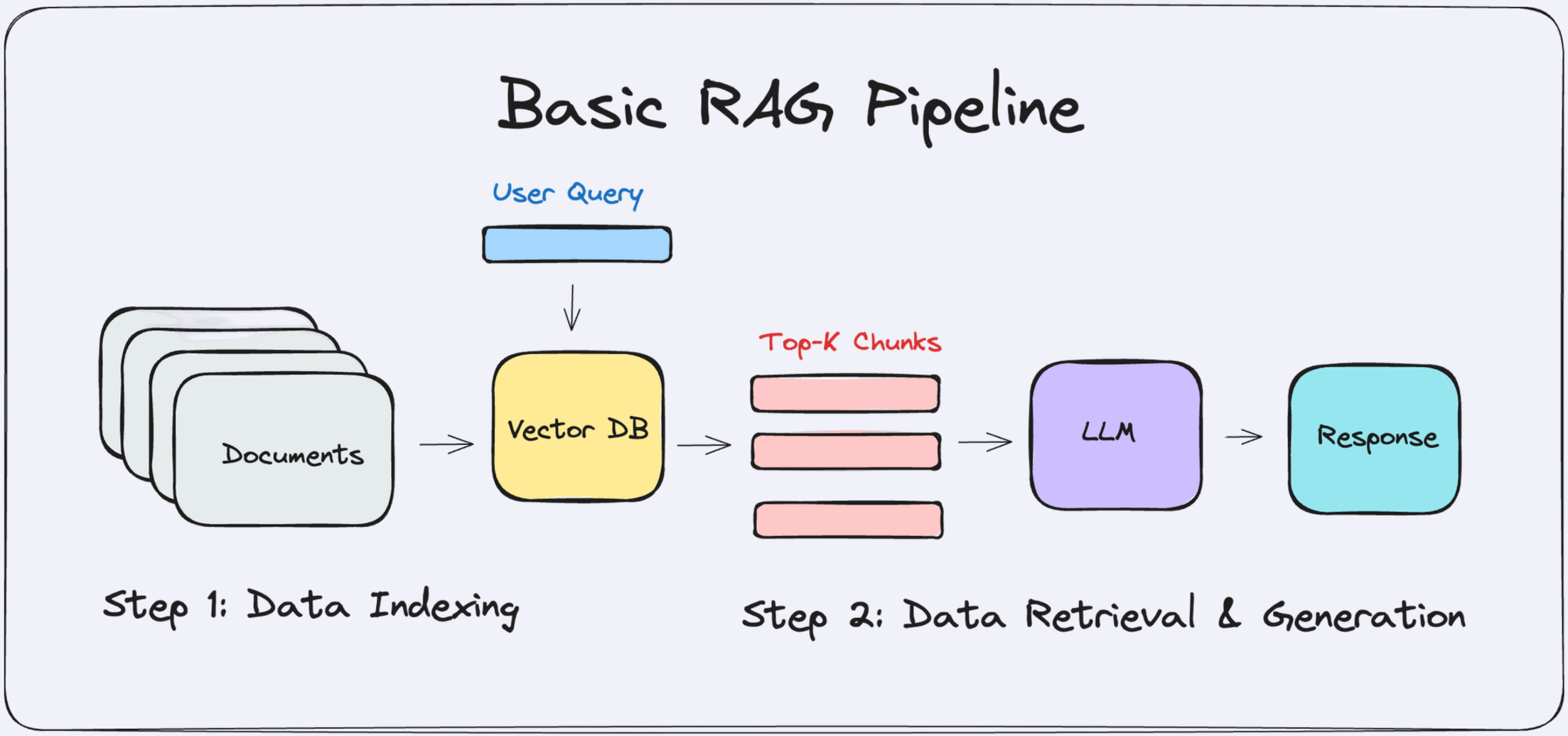
The model answers from retrieved context. *Benefit*: up-to-date, grounded, verifiable answers.

WHEN TO USE:

- **Q&A on private documents**
- **Up-to-date information (post training cutoff)**
- **Reducing hallucinations**
- **Domain-specific knowledge bases**

- ✗ General knowledge questions: just use the LLM
- ✗ Tasks that don't need external knowledge

Basic RAG Pipeline



https://medium.com/@drjulija/what-is-retrieval-augmented-generation-rag-938e4f6e03d1

1. INDEXING

INDEXING (done once):

Documents
↓
Chunking split text into smaller pieces
↓
Embedding transform each chunk into a vector
↓
Vector Store save vectors + original text

Indexing Pipeline

1. **Loading**
   Import the documents that will form your knowledge base.
   Any format: plain text, PDF, web pages, databases.

2. **Chunking**
   Split large documents into smaller pieces.
   Each chunk should be self-contained and meaningful.
   Typical size: 200-500 tokens with 10-20% overlap.

3. **Embedding**
   Transform each chunk into a dense vector using an embedding model.
   Chunks with similar meaning will have similar vectors.
   The embedding model must be the same at indexing and query time.

4. **Storing**
   Save the (vector, original text) pairs in a vector store.
   Done once and reused at every query.

2. QUERYING (Data retrieval and Generation)

QUERYING (done at every question):

User question
↓
Embed question same embedding model as indexing
↓
Similarity search find the most similar chunks
↓
Retrieved context the top-k most relevant chunks
↓
Prompt "Answer using this context: {context}\n{question}"
↓
LLM generates the answer
↓
Answer

Querying Pipeline

1. **Embed the question**
Transform the user question into a vector using the same embedding model used during indexing.
Same model = same vector space = meaningful comparison.

2. **Similarity search**
Find the chunks whose vectors are closest to the question vector in the vector space. Returns the top-k most relevant chunks.

3. **Build the prompt**
Combine the retrieved chunks with the original question into a structured prompt:
"Answer using only this context: {chunks}\n{question}"

4. **Generate**
Pass the augmented prompt to the LLM. The model answers based on the retrieved context, not from its training data alone.

Prompting basics

The key step: combine retrieved context with the question.

Template:

```
"Answer the question using ONLY the provided
context.  If the answer is not in the context,
say 'I don't know.'

Context:  {retrieved_chunks}

Question: {user_question}

Answer:"
```

Why "ONLY the provided context"?
Without this instruction the model might mix retrieved context with its own training knowledge; harder to verify the source of the answer

Why "say I don't know"?
Without this the model will hallucinate an answer even when the context doesn't contain the information: explicit instruction to admit uncertainty

RAG Limitations

Retrieval failures
If the relevant chunk is not retrieved, the model cannot answer correctly.

Chunk boundary problem
The answer might be split across two chunks. Neither chunk alone is sufficient.

Context window limit
More chunks = more context = better recall but also slower and more expensive.

Embedding model quality
A poor embedding model will retrieve irrelevant chunks causing wrong answers.

Hallucination still possible
The model might ignore it and answer from training data.

Evaluation is hard
How do you know if the retrieval was good? How do you measure answer quality at scale?

RAG vs Fine-tuning

USE RAG WHEN:

- Your knowledge changes frequently
- You need to cite sources
- You have private documents
- You want to reduce hallucinations on specific facts

USE FINE-TUNING WHEN:

- You want to change how the model writes
- You need domain-specific vocabulary or tone
- The task format is very different
- Latency matters; no retrieval step

Use both when: You need a model that speaks your domain language AND can answer questions on your documents